



eSigned on **LegalZoom** 62.215.38.249
Fitzgerald 11:39:11 GMT
J. Heslop Sep 22, 2026

```

import A5O
import Closure
import Deployment
import SyntheticNecessity
set_option autoImplicit false

```

```

/-! # A5O — Bridge: v5 checkers ↔ synthetic third-party readings.

```

```

Imports, unmodified: A5O.lean v4 (0e5ebe3f...), Closure.lean (af6b3233...),
v5 Deployment.lean (e07c69ae...), SyntheticNecessity.lean (6263ac8b...).

```

v5's `EvidenceSpec` leaves the six meanings abstract and says so:
 "callers could still choose overly weak predicates." This file removes
 that freedom. `specOf obs` fixes the spec to the six DERIVED readings over
 a `Synthetic.Observable` — what a third party can compute from public
 inputs — so `CheckersSound (specOf obs) v` is soundness against the
 observable, not against a caller's choice.

In particular the receipt's `mandate` boolean is identified with `dM`:
 a sound mandate checker returns `true` only if some public ledger grant
 for that agent NAMES that act. From that, an ALLOW never issues against a
 grant naming nothing (`allow\_requires\_named\_act`) — the seam in
 SyntheticNecessity's second commitment, closed on the rail by
 construction and stated here as a theorem over the checkers.

Round trip: under one named coherence property of the resolver —
 the public binding resolves to the appointing entry — v5's
 `Accountable` over the derived O and `ThirdPartyAccountable` are
 equivalent (`coherent\_equiv`). `Coherent` is a property of the deployed
 resolver (appointment_ref = the grant the decision was made against),
 same object-kind as `NoSideChannel`: attested over a trace, not proved. -/

```

namespace Bridge
open Apertures Synthetic

```

```

abbrev Obs := Synthetic.Observable Principal Agent Action

```

```

/-- The six meanings, fixed by the observable. -/
def specOf (obs : Obs) : EvidenceSpec :=
  { C := dC obs, I := dI obs, M := dM obs, S := dS obs, R := dR obs, Appointed := dApp obs }

```

```

-- ---- Identification of the mandate boolean -----

```

```

/-- A sound mandate checker's `true` means: a public ledger grant for this

```

agent names this act. -/

```
theorem mandate_means_named (obs : Obs) (v : Checkers)
  (hs : CheckersSound (specOf obs) v) (a : Agent) (x : Action)
  (h : v.mandate a x = true) :
  ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.ledger g = some e ∧ e.agent = a ∧ e.names x :=
  hs.mandate a x h
```

/-- A sound scope checker's `true` means: a public ledger grant for this agent bounds this act. Distinct field, distinct fact. -/

```
theorem scope_means_bounded (obs : Obs) (v : Checkers)
  (hs : CheckersSound (specOf obs) v) (a : Agent) (x : Action)
  (h : v.scope a x = true) :
  ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.ledger g = some e ∧ e.agent = a ∧ e.bound x :=
  hs.scope a x h
```

/-- ON THE RAIL THERE IS NO BLANKET GRANT: an ALLOW decision entails a public grant that names the act. -/

```
theorem allow_requires_named_act (obs : Obs) (v : Checkers)
  (hs : CheckersSound (specOf obs) v) (q : Request)
  (h : decision v q = true) :
  ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.ledger g = some e ∧ e.agent = q.agent ∧ e.names q.action :=
  (decision_sound (specOf obs) v hs q h).2.2.1
```

/-- Contrapositive, in v5's own denial form: no named act, no ALLOW. -/

```
theorem unnamed_act_denied (obs : Obs) (v : Checkers)
  (hs : CheckersSound (specOf obs) v) (q : Request)
  (h : ¬ ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.ledger g = some e ∧ e.agent = q.agent ∧ e.names q.action) :
  decision v q = false :=
  no_mandate_denied (specOf obs) v hs q h
```

-- ---- v5 trace accountability lands on the derived O -----

/-- v5's closure, with the spec fixed by the observable. -/

```
theorem trace_accountable_observable (obs : Obs) (v : Checkers) (d : Deployment)
  (hs : CheckersSound (specOf obs) v) (hd : DecisionsFollowChecker v d)
  (hc : NoSideChannel d) :
  Accountable (dC obs) (dI obs) (dM obs) (dS obs) (dR obs) (dApp obs)
  (Executes d) inAoC :=
  trace_accountable (specOf obs) v d hs hd hc
```

-- ---- Round trip under resolver coherence -----

/-- The public binding resolves to the appointing entry: whenever P's pre-execution grant of A over X exists and the act binds to P, the entry the resolver returns IS such a grant. A property of the deployed resolver (appointment_ref = grant decided against). -/

```
def Coherent (obs : Obs) : Prop :=
  ∀ (P : Principal) (A : Agent) (X : Action) (t : Nat),
    dApp obs P A X t → dC obs P A X t →
    ∃ (g : Nat) (e : Grant Principal Agent Action),
      obs.resolve A X t = some g ∧ obs.ledger g = some e ∧
      e.principal = P ∧ e.agent = A ∧ e.names X ∧ e.bound X ∧ e.made < t
```

```
theorem O_implies_authorized (obs : Obs) (hco : Coherent obs)
  (P : Principal) (A : Agent) (X : Action) (t : Nat)
  (hO : Od obs P A X t) : Authorized obs A X t := by
  obtain ⟨hC, hI, _, _, hR, hApp⟩ := hO
  obtain ⟨g, e, hres, hled, hp, hag, hn, hb, hmade⟩ := hco P A X t hApp hC
  refine ⟨g, e, hres, hled, ?_, hag, hn, hb, hmade, hR g e hres hled hp⟩
  rw [hp]; exact hI
```

/-- Under coherence, Closure's `Accountable` over the derived O and the independent `ThirdPartyAccountable` coincide. -/

```
theorem coherent_equiv (obs : Obs) (hco : Coherent obs)
  (Ex : Agent → Action → Nat → Prop) :
  Accountable (dC obs) (dI obs) (dM obs) (dS obs) (dR obs) (dApp obs) Ex inAoC ↔
  ThirdPartyAccountable obs inAoC Ex := by
  constructor
  · intro h A X t d hd he
    obtain ⟨P, hO⟩ := h A X t d hd he
    exact O_implies_authorized obs hco P A X t hO
  · intro h A X t d hd he
    exact authorized_implies_O obs A X t (h A X t d hd he)
```

/-- The full chain: a coherent observable, checkers sound against it, decisions following the checkers, and no side channel, give third-party accountability of the trace — the independent predicate. -/

```
theorem trace_third_party_accountable (obs : Obs) (hco : Coherent obs)
  (v : Checkers) (d : Deployment)
  (hs : CheckersSound (specOf obs) v) (hd : DecisionsFollowChecker v d)
  (hc : NoSideChannel d) :
  ThirdPartyAccountable obs inAoC (Executes d) :=
  (coherent_equiv obs hco (Executes d)).mp
  (trace_accountable_observable obs v d hs hd hc)
```

end Bridge

```
#print axioms Bridge.mandate_means_named
#print axioms Bridge.scope_means_bounded
#print axioms Bridge.allow_requires_named_act
#print axioms Bridge.unnamed_act_denied
#print axioms Bridge.trace_accountable_observable
#print axioms Bridge.O_implies_authorized
#print axioms Bridge.coherent_equiv
#print axioms Bridge.trace_third_party_accountable
```